



COMPLETE TECHNICAL & PRODUCT HANDBOOK

# Qyvox

## Zero-Knowledge Verifiable Identity Engine

*Prove the fact. Keep the data.*

---

A beginner-friendly but implementation-grounded guide for product leaders, investors, engineers, security reviewers, auditors, and enterprise evaluators.

IMPLEMENTATION BASELINE / 27 AUGUST 2026 / VERSION 1.0

## DOCUMENT GUIDE

## How to use this handbook

This handbook is deliberately written in layers. A reader with no cryptography background can begin with the executive overview and the plain-language primer. A technical reader can move directly to the circuit, API, database, test, and deployment chapters. A security reviewer should concentrate on the trust boundaries, threat model, failure behavior, limitations, and production roadmap.

### Reading paths by audience

| Reader                                  | Recommended path                                         |
|-----------------------------------------|----------------------------------------------------------|
| New to zero knowledge                   | Chapters 1-8, then the glossary and FAQ.                 |
| Investor or grant reviewer              | Chapters 1-4, 24-26, 33-35, and the validation snapshot. |
| Frontend engineer                       | Chapters 5-8, 14-16, 22, 28-31.                          |
| Backend or platform engineer            | Chapters 5-8, 17-23, 28-32.                              |
| Cryptography engineer or auditor        | Chapters 9-13 and 24-27, then source appendices.         |
| Enterprise buyer or compliance reviewer | Chapters 3-4, 17-27, 33-36, and the FAQ.                 |

**BASELINE VERSUS PRODUCTION** Every section distinguishes what exists in the repository today from what Qyvox must add before it should be marketed as an issuer-backed, production identity network.

### Document control and accuracy boundary

- Source of truth: the Qyvox repository located at the project workspace on 27 August 2026.
- Credentials: no Supabase key, bearer token, password, ceremony entropy, or private environment value is reproduced in this document.
- Legal posture: privacy and compliance commentary is technical guidance, not legal advice.
- Cryptographic posture: Groth16 security is computational and assumption-based; this guide avoids claims of absolute or information-theoretic secrecy.
- Benchmark posture: timing results are environment-specific observations, not service-level guarantees or mobile-device claims.

**NAVIGATION****Contents**

The editable Word edition uses real heading styles and internal links. Select any chapter below to jump to it. Page numbering may be refreshed in Microsoft Word after future edits.

**PART I / UNDERSTANDING THE PROBLEM**

- 01 Executive overview
- 02 The identity-data problem
- 03 What Qyvox proves
- 04 What Qyvox does not prove

**PART II / ZERO KNOWLEDGE FROM FIRST PRINCIPLES**

- 05 A zero-knowledge mental model
- 06 Arithmetic circuits and R1CS
- 07 Groth16 lifecycle
- 08 Trusted setup and security assumptions

**PART III / THE QYVOX CIRCUIT**

- 09 Circuit specification
- 10 Line-by-line circuit walkthrough
- 11 Boundary conditions and negative tests
- 12 Artifacts and version binding

**PART IV / BROWSER PROVING AND USER EXPERIENCE**

- 13 Frontend architecture
- 14 Private input lifecycle
- 15 Proof generation in the browser
- 16 Execution receipt and developer playground

**PART V / API, AUTHENTICATION, AND PERSISTENCE**

- 17 API contract
- 18 Authentication and authorization
- 19 Verification pipeline
- 20 Replay protection
- 21 Database schema and row-level security
- 22 Status, CORS, and failure semantics

**PART VI / SECURITY MODEL AND PRODUCTION HARDENING**

- 23 Threat model
- 24 Attack analysis and current controls
- 25 Secrets, dependencies, and supply-chain security
- 26 Privacy, governance, and compliance posture
- 27 Production architecture roadmap

**PART VII / BUILD, RUN, TEST, AND OPERATE**

- 28 Repository map
- 29 Local setup and first proof
- 30 Configuration reference
- 31 Testing and validation
- 32 Deployment, observability, and incident response
- 33 Troubleshooting

**PART VIII / PRODUCT, BUSINESS, AND RESEARCH STRATEGY**

- 34 Product and business value
- 35 Enterprise adoption and integration
- 36 Research and benchmarking program
- 37 Open source, auditability, and trust

**PART IX / REFERENCE APPENDICES**

- A API examples
- B Glossary
- C Frequently asked questions
- D Release and audit checklist
- E Source-of-truth index

**Core promise in one sentence**

**QYVOX** The browser proves that a private birth-year witness satisfies an 18+ threshold, while the server receives only a Groth16 proof and two public signals and stores only a minimal verification receipt.

PART I

# Understanding the problem

Why privacy-preserving predicates matter, what Qyvox is, and exactly what the current baseline can and cannot claim.

- 
- 01 Executive overview
  - 02 The identity-data problem
  - 03 What Qyvox proves
  - 04 What Qyvox does not prove

## CHAPTER 01 / QYVOX IMPLEMENTATION GUIDE

## Executive overview

A ten-minute orientation to the product, architecture, and honest security boundary.

Qyvox is a zero-knowledge age-eligibility demonstrator. A user enters a birth year into a Next.js interface. The browser constructs a witness and generates a Groth16 proof with WebAssembly and SnarkJS. The birth year is never placed in the application request. FastAPI authenticates the requester, checks the public statement, rejects exact proof replays, verifies the proof against a pinned verification key, and writes a privacy-minimal receipt to Supabase PostgreSQL.

The key product idea is data minimization by architecture. Many businesses need a yes/no policy decision, such as “is this person at least eighteen?”, but collect the underlying personal fact anyway. Qyvox demonstrates how a verifier can receive evidence of a predicate instead of receiving the raw witness. This shifts the server from being a custodian of a sensitive fact to being a verifier of a cryptographic statement.

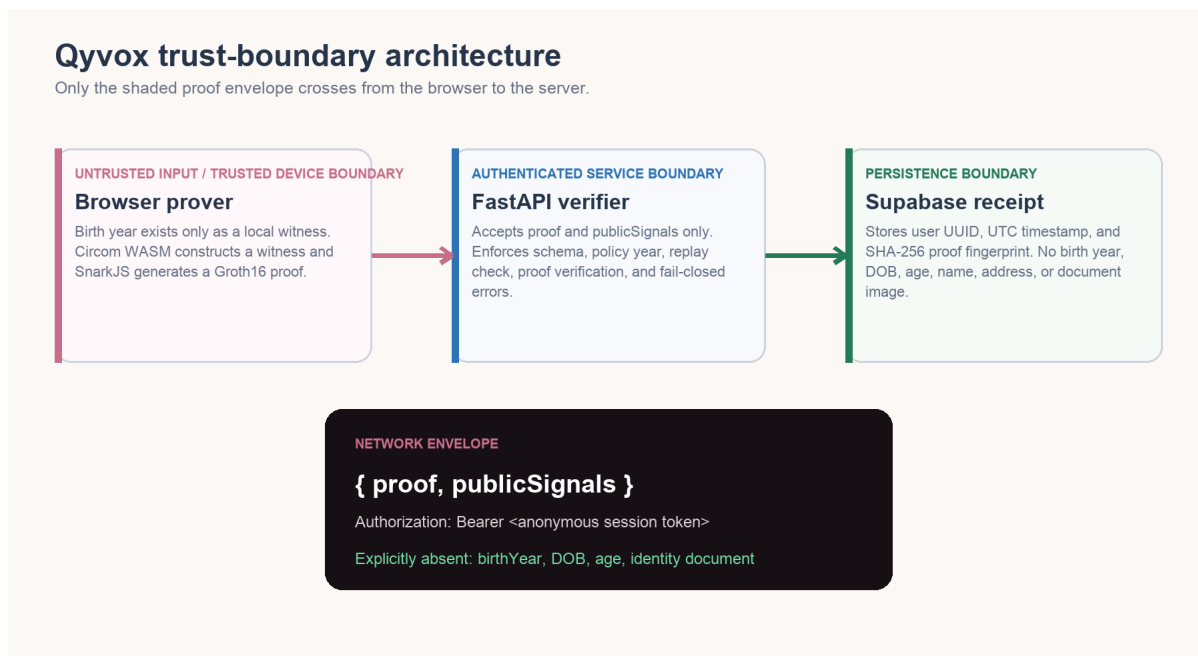


Figure 1. Qyvox trust-boundary architecture

## The current implementation at a glance

### Qyvox implementation stack

| Layer          | Technology                             | Responsibility                                                                                                     |
|----------------|----------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| Circuit        | Circom 2 + circomlib                   | Constrains 16-bit years and forces currentYear - birthYear to be at least 18.                                      |
| Local prover   | SnarkJS + WebAssembly                  | Generates Groth16 proof inside the browser from private and public inputs.                                         |
| Web portal     | Next.js 16 + React 19 + Tailwind CSS 4 | Explains the system, collects the private witness, displays the execution receipt, and exposes developer material. |
| API            | FastAPI + Pydantic                     | Authenticates, validates, checks replay, invokes verification, persists a minimal receipt, and returns a trace.    |
| Authentication | Supabase Auth                          | Creates or restores a PII-free anonymous session and resolves the server-side user UUID.                           |
| Database       | Supabase PostgreSQL                    | Stores user_id, verified_at, and proof_hash only; RLS denies browser access.                                       |
| Proof system   | Groth16 on BN254 / bn128               | Produces a succinct non-interactive proof with a circuit-specific trusted setup.                                   |

**MOST IMPORTANT QUALIFICATION** The baseline proves knowledge of a qualifying number. It does not prove that the number is true or was issued by a government, bank, school, or other trusted authority.

## CHAPTER 02 / QYVOX IMPLEMENTATION GUIDE

### The identity-data problem

Why conventional verification creates unnecessary custody, correlation, and breach risk.

Traditional age checks often copy more information than the business decision requires. A merchant may need only an eligibility bit but receive a full document image, name, date of birth, address, document number, face image, and metadata. Once copied, those values become assets that must be encrypted, access-controlled, retained, deleted, audited, disclosed after breaches, and protected from insiders and secondary use.

Encryption at rest and in transit are necessary controls, but they do not eliminate collection. Authorized applications and operators must still decrypt data to use it. Backups, exports, support systems, analytics pipelines, logs, and incident tooling can widen the exposure surface. Data minimization asks a more fundamental question: can the system make the decision without taking custody of the sensitive fact at all?

## Data-custody comparison

| Conventional model                              | Qyvox baseline                                                                                                    |
|-------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| Server receives a birth year or document.       | Server receives proof and public signals only.                                                                    |
| Database may become a demographic honeypot.     | Receipt table has no DOB, age, name, address, or image field.                                                     |
| Operational staff may access raw identity data. | Routine verification path has no raw witness to retrieve.                                                         |
| Breach can expose reusable identity facts.      | Breach exposes verification receipts and account identifiers, which remain sensitive but are materially narrower. |
| Policy logic is evaluated on disclosed data.    | Policy logic is compiled into constraints and proved locally.                                                     |

## What data minimization changes

- Collection risk falls because the application request has no raw birth year field.
- Retention scope narrows because the database schema cannot store the omitted field without an explicit migration.
- Secondary-use opportunities shrink because there is no raw age attribute in the receipt table.
- Incident impact can be reduced, although account UUIDs, timestamps, and proof fingerprints still require protection.
- Compliance work may become simpler, but lawful basis, notices, access rights, retention rules, and sector obligations still apply.

**PRINCIPLE** The safest sensitive datum is often the datum the server never receives.

## CHAPTER 03 / QYVOX IMPLEMENTATION GUIDE

## What Qyvox proves

The exact statement, public disclosure, and decision that a valid proof supports.

A valid baseline proof supports this statement: “I know a 16-bit integer birthYear such that currentYear - birthYear is at least 18, and the public currentYear is the value included in this proof.” The circuit additionally forces the public output isEligible to equal one. The API accepts only that eligible branch and independently requires currentYear to match the server’s current UTC year.

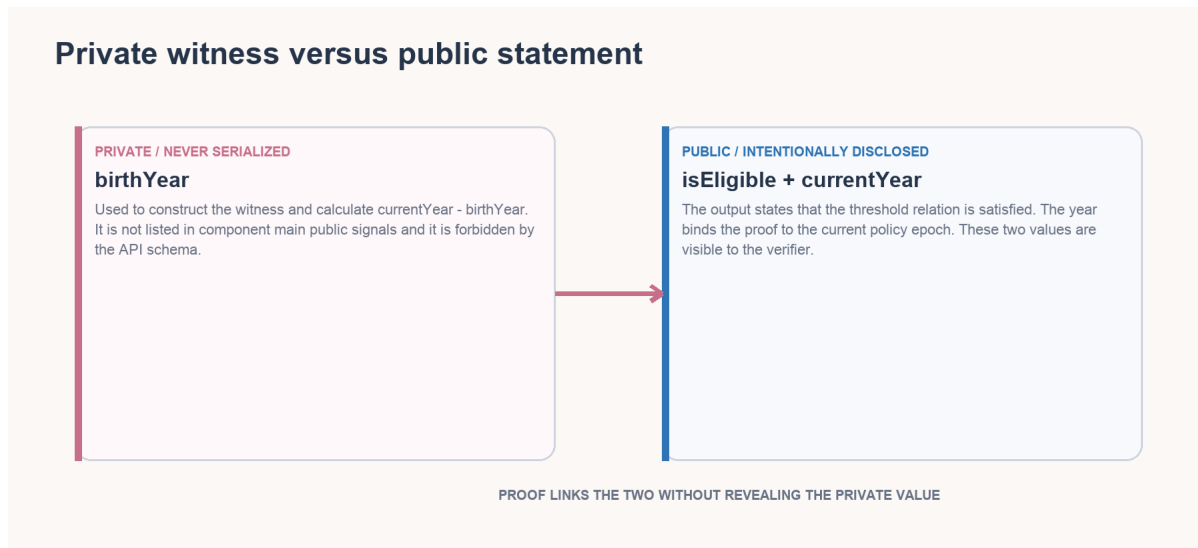


Figure 2. Private witness and public statement

## Meaning of each value

### Signal and artifact dictionary

| Signal or artifact   | Visibility                    | Meaning                                                                                        |
|----------------------|-------------------------------|------------------------------------------------------------------------------------------------|
| birthYear            | Private witness               | The integer used locally to satisfy the age constraint. It is not serialized into the request. |
| currentYear          | Public input                  | The policy epoch. The server requires it to equal its UTC year.                                |
| isEligible           | Public output                 | Must be exactly 1; the circuit cannot produce an accepted underage branch.                     |
| proof                | Public cryptographic artifact | Elliptic-curve points that a verifier checks against the public signals and verification key.  |
| verification receipt | Persistent server record      | Authenticated user UUID, verification time, and exact-proof fingerprint.                       |

The proof is non-interactive: after it is generated, the verifier does not need to question the prover in multiple rounds. It is succinct: the proof object is much smaller than the witness or a general execution trace. It is zero knowledge under the scheme's assumptions: the proof is designed not to disclose the private witness beyond what can be inferred from the public statement.

**WHAT IS INTENTIONALLY REVEALED** The verifier learns that the threshold statement is true for the disclosed policy year. Zero knowledge does not mean “the verifier learns nothing”; it means the verifier learns no additional witness information beyond the public statement under the stated assumptions.

**CHAPTER 04 / QYVOX IMPLEMENTATION GUIDE**

## What Qyvox does not prove

The limitations that must remain visible in product, security, and sales conversations.

---

The current circuit accepts any qualifying integer supplied by the user. It has no issuer signature, credential commitment, document authenticity check, biometric binding, revocation registry, or hardware-backed key. Therefore, it proves knowledge of a satisfying witness, not the truth or provenance of that witness. A user can type a false year and, if it satisfies the relation, generate a mathematically valid proof.

The arithmetic is year-granular. Someone born late in a year is treated the same as someone born early in that year. Exact legal-age rules generally require month/day logic, jurisdiction-specific definitions, leap-year handling, and sometimes time-zone policy. Those are not modeled by this baseline.

### Current boundaries

- No trusted issuer: the witness is self-asserted.
- No exact date: eligibility uses calendar-year subtraction.
- No cryptographic challenge binding: exact replay is blocked by storage, but a next-generation circuit should bind a server nonce, session commitment, and expiry.
- Development-only ceremony: `setup.sh` performs a one-machine ceremony and is not production trust evidence.
- No formal secure-memory erasure: JavaScript state and references are cleared promptly, but managed runtimes do not guarantee physical zeroization.
- No universal compliance guarantee: deployment obligations remain contextual.
- No blockchain requirement or ledger-based consensus: verification runs on ordinary web infrastructure.

## Security-claim language guide

| Claim           | Safe wording                                                                           | Unsafe wording                                                     |
|-----------------|----------------------------------------------------------------------------------------|--------------------------------------------------------------------|
| Witness privacy | Recovering the witness should be computationally infeasible under Groth16 assumptions. | It is mathematically impossible to recover any information.        |
| Identity truth  | The baseline proves knowledge of a qualifying value.                                   | Qyvox proves the user's real government-certified age.             |
| Replay          | Exact proof reuse is rejected; challenge binding is planned.                           | The current circuit is fully replay and front-running resistant.   |
| Compliance      | Data minimization can reduce exposure.                                                 | The architecture automatically guarantees GDPR/KYC/AML compliance. |
| Performance     | Measured on a stated machine and sample size.                                          | Always verifies in under 10 ms everywhere.                         |

PART II

# Zero knowledge from first principles

A plain-language path from statements and witnesses to arithmetic circuits, R1CS, elliptic curves, Groth16, and trusted setup.

---

01 Mental model

02 Arithmetic circuits and R1CS

03 Groth16 lifecycle

04 Trusted setup and assumptions

## CHAPTER 05 / QYVOX IMPLEMENTATION GUIDE

## A zero-knowledge mental model

The vocabulary a first-time reader needs before looking at code.

Imagine a locked rulebook that says which secret inputs are acceptable. A prover wants to convince a verifier that one acceptable input exists and that the prover knows it, without opening the secret page. The prover runs a cryptographic algorithm using the secret witness and public statement. The verifier checks the resulting proof using public verification material.

### Zero-knowledge vocabulary

| Term             | Plain-language meaning                                              | Qyvox example                                                                 |
|------------------|---------------------------------------------------------------------|-------------------------------------------------------------------------------|
| Statement        | The claim the verifier is willing to accept.                        | Eligibility is true for the current year.                                     |
| Witness          | Private values that make the statement true.                        | birthYear.                                                                    |
| Circuit          | A fixed set of mathematical relations that defines valid witnesses. | Range checks plus $\text{currentYear} - \text{birthYear} \geq 18$ .           |
| Prover           | Algorithm that uses the witness to create a proof.                  | SnarkJS <code>groth16.fullProve</code> in the browser.                        |
| Proof            | Succinct public evidence checked without seeing the witness.        | The <code>pi_a</code> , <code>pi_b</code> , <code>pi_c</code> Groth16 object. |
| Verifier         | Algorithm that checks proof + public inputs.                        | SnarkJS <code>groth16.verify</code> launched by FastAPI.                      |
| Verification key | Public material tied to a particular circuit setup.                 | <code>backend/zk/verification_key.json</code> .                               |

### Three security properties

- **Completeness:** an honest prover with a valid witness should be able to produce a proof the verifier accepts.
- **Soundness or knowledge soundness:** a prover should not be able to convince the verifier without a witness that satisfies the circuit, except with negligible probability under the scheme's assumptions.
- **Zero knowledge:** the proof should reveal no useful information about the private witness beyond the public statement, under the scheme's assumptions and correct implementation.

**IMPORTANT DISTINCTION** A circuit can be cryptographically sound and still encode the wrong business rule. Circuit review must check both mathematics and policy semantics.

## CHAPTER 06 / QYVOX IMPLEMENTATION GUIDE

## Arithmetic circuits and R1CS

How a business rule becomes field equations that a proving system can enforce.

Circom programs do not execute like ordinary JavaScript. They describe constraints over a finite field. The compiler turns signals and component relations into a Rank-1 Constraint System (R1CS). Each constraint has the conceptual form  $\langle A, w \rangle \times \langle B, w \rangle = \langle C, w \rangle$ , where  $w$  is the full assignment of constants, public signals, private signals, and intermediate values.

A witness calculator evaluates the circuit for a particular input and produces candidate signal values. A valid witness must satisfy every R1CS equation. The proving algorithm then creates an argument that those equations are satisfied without disclosing the private components of  $w$ .

### Why comparisons need special gadgets

Finite fields do not have an intrinsic “greater than” operator. Comparisons are implemented by constraining bit decompositions and arithmetic relations. Qyvox uses circomlib’s Num2Bits and GreaterEqThan components with a 16-bit width. The explicit bit range is security-relevant: without it, finite-field subtraction can wrap and a future birth year might be represented as a large field element that appears to exceed eighteen.

#### Circuit-safety controls

| Circuit concern        | Control in Qyvox                                                                                                                |
|------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| Year range             | Num2Bits(16) constrains both birthYear and currentYear.                                                                         |
| Future-year wraparound | The private difference is also decomposed to 16 bits, so negative field representations cannot masquerade as a small valid age. |
| Threshold              | GreaterEqThan(16) compares the constrained difference against 18.                                                               |
| Eligible output        | isEligible is connected to the comparator output and constrained to equal 1.                                                    |
| Public binding         | currentYear is declared public; output isEligible is public by Circom convention.                                               |

**R1CS LESSON** “Calculated” is not the same as “constrained.” Every security-relevant relationship must be enforced by constraints, not merely assigned in witness-generation code.

## CHAPTER 07 / QYVOX IMPLEMENTATION GUIDE

## Groth16 lifecycle

What the proving key, verification key, proof points, and pairing check do at a conceptual level.

---

Groth16 is a succinct non-interactive argument system for arithmetic circuits. During setup, circuit constraints are transformed into structured public parameters. The proving key is used with a full witness to create proof elements. The verification key is used with public signals to evaluate pairing-based equations on the BN254 curve. A successful pairing equation convinces the verifier that a satisfying witness exists, subject to the setup and cryptographic assumptions.

1. **Compile.** Circom produces R1CS, a WebAssembly witness generator, and a symbol map.
2. **Prepare.** A Powers of Tau transcript provides universal structured randomness sized above the circuit's constraint count.
3. **Specialize.** Groth16 setup binds parameters to the exact age\_check R1CS.
4. **Contribute.** Additional circuit-specific entropy is mixed into the zkey.
5. **Export.** The verification key is derived from the final zkey.
6. **Prove.** The browser combines the private witness, public input, WASM calculator, and proving key.
7. **Verify.** The server checks proof, public signals, and verification key.

The proof object serialized by SnarkJS contains three groups of elliptic-curve coordinates: `pi_a`, `pi_b`, and `pi_c`, plus protocol and curve labels. The API validates the JSON shape and decimal-scalar syntax before invoking the cryptographic verifier. Shape validation is not proof validation; it merely prevents malformed or unbounded inputs from reaching deeper layers.

**CURVE NAMING** The repository and SnarkJS use `bn128`; the handbook also refers to `BN254`. These names commonly refer to the same pairing-friendly curve family in this tooling context.

## CHAPTER 08 / QYVOX IMPLEMENTATION GUIDE

## Trusted setup and security assumptions

Why the ceremony matters and how to describe it without either panic or hand-waving.

---

Groth16 requires a circuit-specific structured reference string. Setup participants temporarily know secret randomness sometimes described as toxic waste. If every participant colludes and retains enough secret material, soundness can be compromised. A multi-party computation ceremony mitigates this: security holds if at least one contributor generates its randomness honestly and destroys it.

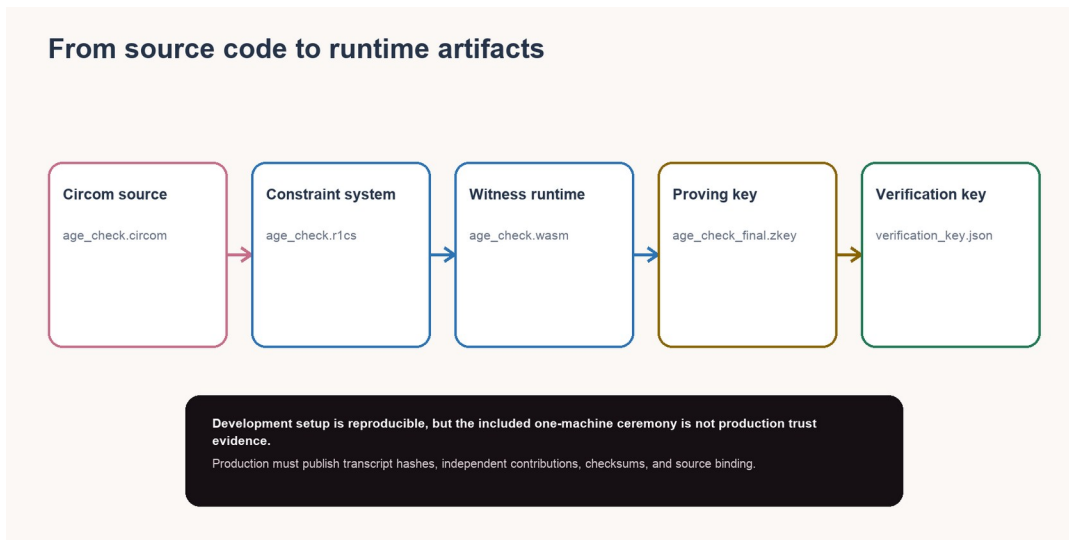


Figure 3. Circuit source and artifact pipeline

## What setup.sh actually does

- Enables strict shell error handling and a restrictive umask.
- Checks that Circom, OpenSSL, and the pinned SnarkJS executable are available.
- Compiles age\_check.circom to R1CS, WASM, and symbol outputs.
- Creates a bn128 Powers of Tau transcript at power 12, sufficient for up to  $2^{12}$  constraints.
- Adds local random entropy, prepares phase two, performs circuit-specific Groth16 setup, and contributes again.
- Verifies the final zkey, exports the verification key, and copies runtime artifacts to frontend/public/zk and backend/zk.

## Production ceremony requirements

- Use an independently reviewed public Powers of Tau transcript.
- Publish the exact R1CS hash and source revision before contributions begin.
- Accept independent contributors using separated machines and documented procedures.
- Publish contribution hashes, verification logs, software versions, and final artifact checksums.
- Apply a public random beacon when appropriate and record the derivation.
- Have external cryptography reviewers verify that deployed artifacts match the published ceremony outputs.

**CURRENT STATUS** The one-machine setup script is valuable for development reproducibility. It must never be represented as proof that production toxic waste was independently destroyed.

PART III

# The Qyvox circuit

A line-by-line explanation of signals, range constraints, comparison logic, public ordering, artifacts, and negative tests.

---

01 Circuit specification

02 Line-by-line walkthrough

03 Boundary conditions and tests

04 Artifacts and version binding

## CHAPTER 09 / QYVOX IMPLEMENTATION GUIDE

## Circuit specification

The formal inputs, outputs, invariant, and public signal ordering of `age_check.circom`.

### Simplified complete circuit listing

```
pragma circom 2.0.0;

include "circomlib/circuits/comparators.circom";
include "circomlib/circuits/bitify.circom";

template AgeCheck(nBits) {
  signal input birthYear;
  signal input currentYear;
  signal output isEligible;
  signal ageInYears;

  component birthYearBits = Num2Bits(nBits);
  component currentYearBits = Num2Bits(nBits);
  component ageBits = Num2Bits(nBits);
  component ageThreshold = GreaterEqThan(nBits);

  birthYearBits.in <== birthYear;
  currentYearBits.in <== currentYear;
  ageInYears <== currentYear - birthYear;
  ageBits.in <== ageInYears;
  ageThreshold.in[0] <== ageInYears;
  ageThreshold.in[1] <== 18;
  isEligible <== ageThreshold.out;
  isEligible == 1;
}

component main { public [currentYear] } = AgeCheck(16);
```

### Age-check circuit contract

| Property                  | Value                                 |
|---------------------------|---------------------------------------|
| Private input             | birthYear                             |
| Explicit public input     | currentYear                           |
| Public output             | isEligible                            |
| Threshold                 | 18 calendar years                     |
| Bit width                 | 16                                    |
| Field / curve environment | BN254 / bn128 through SnarkJS Groth16 |
| Observed compiled size    | 74 constraints and 72 wires           |
| Public signal order       | [isEligible, currentYear]             |

**PUBLIC-OUTPUT ORDERING** Circom exposes output signals before explicitly public input signals. The API therefore interprets `publicSignals[0]` as `isEligible` and `publicSignals[1]` as `currentYear`.

## CHAPTER 10 / QYVOX IMPLEMENTATION GUIDE

### Line-by-line circuit walkthrough

Why each component exists and what can go wrong if it is removed.

---

#### Pragma and imports

The pragma requires the Circom 2 language family. `comparators.circom` provides `GreaterEqThan`; `bitify.circom` provides `Num2Bits`. Imports are resolved from the pinned `circomlib 2.0.5` dependency. Pinning avoids silently changing constraint implementations when dependencies update.

#### Signals and data flow

`birthYear` and `currentYear` are inputs to the template. Privacy is determined at the main component declaration, not by the word “private” attached to a signal. `currentYear` is explicitly listed as public; `birthYear` is omitted and therefore remains private. `isEligible` is a signal output and becomes public. `ageInYears` is an internal witness signal.

#### Range constraints

`Num2Bits(16)` constrains a value to a 16-bit non-negative integer representation. Qyvox applies this to both years and to their difference. This is not cosmetic input validation. It turns the application’s integer assumptions into circuit-enforced facts and closes a finite-field wraparound avenue.

#### Comparison and forced outcome

`GreaterEqThan(16)` receives `ageInYears` and the constant 18. Its output is one when the first is at least the second. The assignment `isEligible <== ageThreshold.out` constrains the output to the comparator result. The final equality `isEligible == 1` ensures that only an eligible witness can satisfy the circuit. Without the final equality, the circuit could prove either branch and ask the server to interpret the output; Qyvox intentionally makes ineligibility unsatisfiable.

**REVIEW QUESTION** Can every security-relevant signal float freely? For each value, trace both its witness assignment and the constraints that force it to be correct.

## CHAPTER 11 / QYVOX IMPLEMENTATION GUIDE

### Boundary conditions and negative tests

How the repository demonstrates behavior at the threshold and under adversarial inputs.

---

**Circuit smoke-test matrix**

| Case                     | Input                             | Expected result                  | Reason                                                          |
|--------------------------|-----------------------------------|----------------------------------|-----------------------------------------------------------------|
| Clearly eligible         | 2000 / 2026                       | Proof valid; signals [1, 2026]   | Age difference is 26.                                           |
| Exact boundary           | 2008 / 2026                       | Proof valid                      | Difference is exactly 18.                                       |
| Underage                 | 2010 / 2026                       | Witness/proof generation rejects | Difference is 16; isEligible cannot equal 1.                    |
| Future year              | 2027 / 2026                       | Rejects                          | Negative difference cannot fit the constrained 16-bit range.    |
| Out-of-range public year | 65517 / 65536                     | Rejects                          | 65536 does not fit in 16 bits.                                  |
| Tampered public signal   | Valid proof, year changed to 2027 | Verification false               | Proof is cryptographically bound to the original public signal. |

The smoke test also invokes the exact backend/verify.mjs helper used by FastAPI, providing an integration bridge between circuit artifacts and the server verifier. The test terminates Node explicitly because SnarkJS worker handles can otherwise keep the event loop alive after assertions complete.

**Additional tests recommended before production**

- Property-based generation across all relevant year boundaries.
- Differential tests against an independent reference implementation of the policy.
- Mutation tests that remove or weaken one constraint at a time and require tests to fail.
- Independent R1CS inspection and unconstrained-signal analysis.
- Fuzzing of public signal order, scalar encodings, and verifier input parsing.
- Cross-browser proof generation on low-memory mobile devices.

**CHAPTER 12 / QYVOX IMPLEMENTATION GUIDE****Artifacts and version binding**

How source, R1CS, WASM, proving key, and verification key must remain one coherent release.

A proof generated from one circuit setup cannot be treated as interchangeable with another. The R1CS defines the relation. The WASM constructs a witness for that relation. The proving key is circuit-specific. The verification key is derived from the same final setup. Deploying a mismatched combination results in failure—or, worse, in confusing provenance if releases are not labeled.

## Cryptographic artifact inventory

| Artifact              | Consumer                       | Repository location       | Release control                                |
|-----------------------|--------------------------------|---------------------------|------------------------------------------------|
| age_check.circom      | Circuit compiler and reviewers | circuits/age_check.circom | Source commit and code review.                 |
| age_check.r1cs        | Setup and auditors             | circuits/build            | Publish hash and circuit statistics.           |
| age_check.wasm        | Browser prover                 | frontend/public/zk        | Immutable asset hash and cache policy.         |
| age_check_final.zkey  | Browser prover                 | frontend/public/zk        | Ceremony transcript binding and checksum.      |
| verification_key.json | Backend verifier               | backend/zk                | Pinned image/release hash.                     |
| age_check.sym         | Developers and auditors        | circuits/build            | Debug-only mapping; publish with audit bundle. |

## Recommended release manifest

## Illustrative release-manifest shape

```
{
  "circuit": "age_check",
  "version": "1.0.0",
  "source_commit": "<git commit>",
  "r1cs_sha256": "<64 hex characters>",
  "wasm_sha256": "<64 hex characters>",
  "zkey_sha256": "<64 hex characters>",
  "verification_key_sha256": "<64 hex characters>",
  "public_signals": ["isEligible", "currentYear"],
  "ceremony_transcript": "<public URL and digest>"
}
```

**DEPLOYMENT INVARIANT** The frontend artifacts and backend verification key must be released atomically from the same verified manifest.

PART IV

# Browser proving and user experience

How the Next.js application constructs the proof locally, clears private state, authenticates without identity fields, and exposes real execution evidence.

- 
- 01 Frontend architecture
  - 02 Private input lifecycle
  - 03 Proof generation
  - 04 Execution receipt and developer playground

CHAPTER 13 / QYVOX IMPLEMENTATION GUIDE

# Frontend architecture

The routes, components, libraries, assets, and rendering choices that make up the Qyvox portal.

Frontend source map

| Path                              | Role                                                                                 |
|-----------------------------------|--------------------------------------------------------------------------------------|
| frontend/app/page.tsx             | Executive landing page and live proof demo placement.                                |
| frontend/app/whitepaper/page.tsx  | Technical narrative, primitives, trust model, and security bounds.                   |
| frontend/app/docs/page.tsx        | Quickstart, architecture, circuit, SDK patterns, API, schema, and playground.        |
| frontend/app/research/page.tsx    | Benchmark results, methodology boundaries, and validation roadmap.                   |
| frontend/app/faq/page.tsx         | Enterprise due-diligence questions and bounded answers.                              |
| frontend/components/ProofDemo.tsx | Private input form, proof lifecycle, execution receipt, and exact payload inspector. |
| frontend/lib/zkProver.ts          | Dynamic SnarkJS import and browser-only fullProve wrapper.                           |
| frontend/lib/verifyProof.ts       | Anonymous session resolution and proof-only API request.                             |
| frontend/lib/supabaseClient.ts    | Lazy browser client and anonymous authentication.                                    |
| frontend/public/zk/*              | Public WASM and proving key consumed by the browser.                                 |

The App Router statically prerenders the marketing and technical pages. The proof component is explicitly a client component because it depends on browser state, performance timing, WebAssembly, and dynamic SnarkJS loading. Dynamic import prevents proof code from being executed during server-side rendering.

## Design system

The portal uses a dark ink, ivory, and restrained rose palette; Space Grotesk for display and interface text; and IBM Plex Mono for technical labels. The interface uses responsive grid layouts, smooth but bounded motion, reduced-motion handling, minimum interactive target sizes, semantic landmarks, live regions for proof status, and overflow protections for narrow screens.

**PRODUCT PRINCIPLE** The interface should make the privacy boundary observable: show what stayed local, what crossed the network, what the verifier did, and what was stored.

## CHAPTER 14 / QYVOX IMPLEMENTATION GUIDE

## Private input lifecycle

Every place the birth year exists, and the controls that limit its lifetime.

1. **Form state.** The value begins as a string in React state bound to the numeric input.
2. **Validation copy.** Submit converts the string to a local number and checks integer/range/eligibility conditions.
3. **Proof input.** generateAgeProof receives birthYear and currentYear and converts them to decimal strings for SnarkJS.
4. **Witness calculation.** The WASM calculator uses the birth year inside browser memory.
5. **Proof completion.** After fullProve returns, the component clears React state and sets the mutable local variable to zero before authentication or fetch.
6. **Final cleanup.** The finally branch zeroes the local variable and clears state again on both success and error paths.

The outgoing JSON is constructed from the ProofBundle only. verifyProof serializes exactly two top-level keys: proof and publicSignals. TypeScript interfaces document the fixed proof shape. The backend separately rejects unknown top-level fields, so a body containing birthYear fails validation even if a caller bypasses the official frontend.

### What “cleared” means in JavaScript

Clearing state and overwriting a reference shortens the value’s reachable lifetime. It does not provide certified physical memory erasure. Garbage collectors, copies made by browser internals, swap, crash reports, malicious extensions, compromised scripts, screen recording, and operating-system memory inspection remain outside the guarantee. A production privacy statement should say that the application promptly removes references and never serializes the value, not that every physical byte is provably zeroized.

**BROWSER THREAT BOUNDARY** Zero knowledge protects the witness from the verifier. It does not protect a user whose browser, device, supply chain, or loaded JavaScript is already compromised.

## CHAPTER 15 / QYVOX IMPLEMENTATION GUIDE

### Proof generation in the browser

How zkProver.ts loads artifacts, runs fullProve, and validates the public statement before transport.

---

Browser prover control flow (abridged)

```
const snarkjs = await import("snarkjs");
const result = await snarkjs.groth16.fullProve(
  {
    birthYear: birthYear.toString(10),
    currentYear: currentYear.toString(10),
  },
  "/zk/age_check.wasm",
  "/zk/age_check_final.zkey",
);
if (
  result.publicSignals.length !== 2 ||
  result.publicSignals[0] !== "1" ||
  result.publicSignals[1] !== currentYear.toString(10)
) throw new Error("Unexpected public statement");
```

The function refuses to run during server rendering and requires safe integers. Decimal strings prevent accidental floating-point notation. After proving, it normalizes public signals to strings and applies a client-side defense-in-depth check. The server repeats the policy check; client validation is never treated as authoritative.

### Public artifact delivery

- WASM and zkey are public cryptographic artifacts, not secrets.
- They should be served with immutable versioned URLs or content hashes in production.
- Subresource integrity, a strict Content Security Policy, dependency pinning, and build provenance reduce supply-chain risk.
- Larger future circuits should move proving to a Web Worker to keep the interface responsive.
- Cold-download, cache-hit, memory, and thermal behavior must be measured separately on mobile devices.

**DO NOT CONFUSE PUBLIC WITH UNIMPORTANT** The proving key is intentionally public, but an attacker who replaces it or the WASM can alter availability or trick the user interface. Artifact integrity still matters.

## CHAPTER 16 / QYVOX IMPLEMENTATION GUIDE

### Execution receipt and developer playground

How Qyvox demonstrates that a real proof path ran instead of showing a fabricated animation.

---

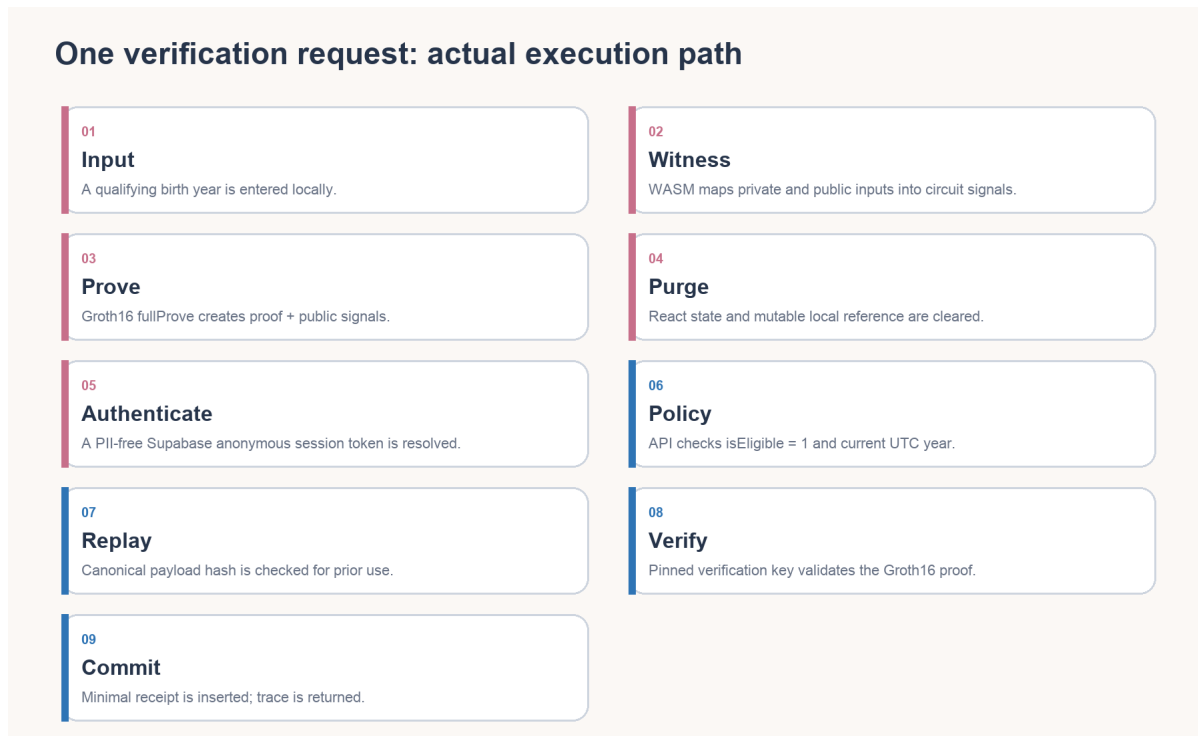


Figure 4. Live verification request lifecycle

ProofDemo records browser events with `performance.now()`, changes stage labels as authentication and transport proceed, and appends server-measured trace steps returned by FastAPI. The server trace names policy, replay, verifier, and database operations and includes non-negative durations. The UI displays the exact request JSON after proof generation, excluding authorization headers and explicitly noting the absence of the witness.

#### Execution-receipt evidence

| Evidence element                          | Source                                                       | What it demonstrates                                                                                       |
|-------------------------------------------|--------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|
| Browser prover duration                   | <code>performance.now()</code> around <code>fullProve</code> | Real work occurred in this tab.                                                                            |
| Auth and round-trip duration              | <code>verifyProof</code> client timing                       | Actual session resolution and network request.                                                             |
| Policy/replay/verifier/database durations | FastAPI <code>perf_counter</code>                            | Server returned trace from this successful request.                                                        |
| Exact payload inspector                   | Serialized <code>ProofBundle</code>                          | Only proof and <code>publicSignals</code> are in the body.                                                 |
| Stored-fields boundary                    | Server trace constant                                        | Intended persistent fields are <code>user_id</code> , <code>verified_at</code> , <code>proof_hash</code> . |

The receipt is observability, not a second proof system. A malicious server could lie about its internal timings or stored fields. Independent assurance requires source review, deployment attestation, database policy review, audit logs, and where appropriate remote attestation or third-party audits. The receipt's value is transparency for a live demo and integration debugging.

PART V

# API, authentication, and persistence

The server's strict request boundary, bearer-token resolution, replay logic, isolated verifier, minimal schema, CORS, and fail-closed behavior.

---

- 01 API contract
- 02 Authentication and authorization
- 03 Verification pipeline
- 04 Replay protection
- 05 Database and RLS
- 06 Status, CORS, and failure semantics

## CHAPTER 17 / QYVOX IMPLEMENTATION GUIDE

## API contract

The exact request/response schema and why strict validation matters.

## Accepted request shape (coordinates abbreviated)

```
POST /api/v1/verify
Authorization: Bearer <Supabase access token>
Content-Type: application/json

{
  "proof": {
    "pi_a": ["...", "...", "1"],
    "pi_b": [["...", "..."], ["...", "..."], ["1", "0"]],
    "pi_c": ["...", "...", "1"],
    "protocol": "groth16",
    "curve": "bn128"
  },
  "publicSignals": ["1", "2026"]
}
```

Pydantic forbids extra fields at both the request and proof levels. Scalar strings must contain one to eighty decimal digits. pi\_a and pi\_c have exactly three elements; pi\_b has exactly three rows with exactly two coordinates. publicSignals has exactly two values. The content-length middleware rejects oversized verify requests before JSON parsing when the header is present, and infrastructure should enforce the same limit for chunked bodies.

## Verifier API error semantics

| Status | Meaning                                          | Example                                                     |
|--------|--------------------------------------------------|-------------------------------------------------------------|
| 200    | Proof accepted and receipt committed             | verified=true with verified_at and trace.                   |
| 400    | Malformed transport metadata                     | Invalid Content-Length.                                     |
| 401    | Missing, invalid, or expired bearer session      | No accepted Supabase user.                                  |
| 403    | Policy statement or cryptographic proof rejected | Stale year, isEligible ≠ 1, or invalid proof.               |
| 409    | Account or exact proof already consumed          | Replay fingerprint or primary-key conflict.                 |
| 413    | Payload exceeds configured bound                 | Content length over MAX_REQUEST_BYTES.                      |
| 422    | JSON shape or scalar validation failed           | birthYear field, wrong proof dimensions, or unknown field.  |
| 502    | Replay or persistence database dependency failed | System fails closed instead of accepting without a receipt. |
| 503    | Verifier unavailable or timed out                | Missing artifact, Node failure, timeout, malformed verdict. |

## CHAPTER 18 / QYVOX IMPLEMENTATION GUIDE

## Authentication and authorization

How Qyvox obtains an account-scoped receipt without collecting identity attributes in the demo.

The browser lazily creates a Supabase client from `NEXT_PUBLIC_SUPABASE_URL` and a publishable key. It first restores an existing session. If none exists, it calls `signInAnonymously`. The resulting access token is attached as a bearer header. The birth year is already cleared before this step.

FastAPI never accepts a client-provided `user_id`. It sends the bearer token to Supabase Auth, requires a user object, and converts the returned identifier to UUID. This prevents a caller from choosing a different receipt owner by editing JSON. The service-role key stays on the backend and must never be prefixed `NEXT_PUBLIC` or shipped to the browser.

### Credential boundary

| Credential                | Location                               | Exposure rule                                                               |
|---------------------------|----------------------------------------|-----------------------------------------------------------------------------|
| Supabase publishable key  | Frontend environment                   | Designed for browser use; still rely on RLS and Auth, not key secrecy.      |
| Anonymous access token    | Browser session + Authorization header | Short-lived bearer credential; protect from XSS and logs.                   |
| Supabase service-role key | Backend environment only               | Secret and privileged; never commit, print, embed, or expose to the client. |
| Ceremony entropy          | Ephemeral setup process                | Generated locally and never written to source or runtime configuration.     |

**CREDENTIAL HYGIENE** Any service-role or secret key pasted into a chat, issue, screenshot, log, or repository should be treated as exposed and rotated. This handbook contains placeholders only.

## CHAPTER 19 / QYVOX IMPLEMENTATION GUIDE

## Verification pipeline

The ordered server controls applied to each authenticated proof.

1. **Request bound.** Reject declared bodies over the configured maximum before parsing.
2. **Schema validation.** Require exact proof dimensions, decimal scalar syntax, protocol, curve, and two public signals.

3. **Authentication.** Resolve the bearer token through Supabase Auth and derive the user UUID.
4. **Policy binding.** Require isEligible = 1 and public year = server current UTC year.
5. **Fingerprint.** Serialize the validated payload canonically and compute SHA-256.
6. **Replay query.** Look up the fingerprint; fail closed if the database query cannot complete.
7. **Concurrency gate.** Acquire a bounded semaphore slot before launching the verifier.
8. **Cryptographic verification.** Invoke verify.mjs with a pinned verification key, memory bound, timeout, fixed working directory, and JSON over stdin.
9. **Persistence.** Insert user UUID, UTC timestamp, and proof hash. Never return success if the record cannot be saved.
10. **Response.** Return verified=true, timestamp, and a privacy-safe timing trace.

The order is intentional. Cheap checks run before expensive verification. Replay detection runs before the verifier to avoid repeated CPU use. Persistence runs after proof validation, and its failure prevents a success receipt. Configuration failures are 503 server errors, not 403 invalid-proof responses, which preserves meaningful operational diagnosis.

**FAIL CLOSED** If replay storage or receipt persistence is unavailable, the API rejects the request. It does not silently skip the control and claim success.

## CHAPTER 20 / QYVOX IMPLEMENTATION GUIDE

### Replay protection

What the current fingerprint control blocks, what it does not block, and the challenge-bound design needed next.

---

The API serializes the validated proof and public signals with sorted keys and compact separators, hashes the bytes with SHA-256, and checks proof\_hash in verified\_users. The column is unique. A repeated canonical payload is rejected before verification, and a racing duplicate insert is caught by the unique constraint.

## Replay and freshness behavior

| Scenario                                           | Current result               | Explanation                                                                              |
|----------------------------------------------------|------------------------------|------------------------------------------------------------------------------------------|
| Identical request resent                           | Blocked                      | Same canonical hash already exists.                                                      |
| Same proof with JSON whitespace changed            | Blocked                      | Server hashes its own canonical model dump, not raw bytes.                               |
| Same valid proof submitted by another account      | Blocked after first use      | Fingerprint uniqueness is global.                                                        |
| New proof generated from same witness              | Not necessarily blocked      | Groth16 proving is randomized; a fresh proof has a new fingerprint.                      |
| Proof copied before first submission and front-run | Not cryptographically solved | Current proof is not bound to a server challenge or intended account inside the circuit. |
| Stale public year                                  | Blocked                      | Policy check requires current server UTC year.                                           |

## Next-generation challenge binding

A production circuit should include public challenge data such as a random server nonce, verifier or relying-party identifier, session commitment, policy identifier, and expiry. The server issues a one-time challenge, the browser proves against it, and the server atomically consumes it. This binds the proof to a session and purpose rather than merely detecting an identical serialized proof after the fact.

**CURRENT ERROR MESSAGE** “Replay protection is temporarily unavailable” means the database lookup failed. It is a 502 dependency failure, not evidence that a proof was invalid.

## CHAPTER 21 / QYVOX IMPLEMENTATION GUIDE

## Database schema and row-level security

Why the persistence model is deliberately small and how Supabase access is constrained.

## Privacy-minimal receipt schema

```
create table public.verified_users (
  user_id uuid primary key references auth.users(id) on delete cascade,
  verified_at timestamptz not null default timezone('utc', now()),
  proof_hash text not null unique
  check (proof_hash ~ '^([0-9a-f]){64}$')
);

alter table public.verified_users enable row level security;
revoke all on table public.verified_users from anon, authenticated;
```

## Persistent data dictionary

| Column      | Purpose                                               | Privacy note                                                          |
|-------------|-------------------------------------------------------|-----------------------------------------------------------------------|
| user_id     | Account-scoped verification identity and primary key. | Pseudonymous identifier, still personal/security-relevant in context. |
| verified_at | UTC time of accepted verification.                    | Can reveal activity timing and must be access-controlled.             |
| proof_hash  | Exact-proof replay fingerprint.                       | One-way fingerprint of public proof payload; not a DOB hash.          |

There is no browser policy granting select, insert, update, or delete. The backend uses the service role to bypass RLS for controlled operations. Anonymous and authenticated roles are explicitly revoked. The live integration test checks that direct browser-role reads and writes are denied and that the persisted row contains exactly the three expected columns.

**PRIVACY NUANCE** A minimal receipt is not “zero data.” UUIDs, timestamps, and proof fingerprints still need retention rules, access control, monitoring, and breach analysis.

## CHAPTER 22 / QYVOX IMPLEMENTATION GUIDE

## Status, CORS, and failure semantics

Operational controls that keep browser-to-API communication narrow and diagnosable.

CORSMiddleware permits only configured exact origins, with GET, POST, and OPTIONS and only Authorization and Content-Type request headers. Wildcards are rejected by the settings parser. This protects browser integrations from accidental broad origin exposure, although CORS is not an authentication control and does not stop non-browser clients.

GET /health reports only that the FastAPI process responds. GET /status performs a bounded dependency check and returns operational or unavailable labels for the verifier artifacts and database. It avoids exposing raw stack traces, credentials, file paths, or database diagnostics to public callers.

**Failure classification**

| Failure                        | External response | Security rationale                                             |
|--------------------------------|-------------------|----------------------------------------------------------------|
| Malformed or oversized request | 400/413/422       | Reject before expensive cryptography.                          |
| Invalid/stale proof            | 403               | No receipt is written.                                         |
| Missing/expired auth           | 401               | No client-chosen identity accepted.                            |
| Replay                         | 409               | Exact proof or already-verified account is not accepted again. |
| Replay database outage         | 502               | Cannot establish non-reuse, so fail closed.                    |
| Verifier runtime outage        | 503               | Do not mislabel infrastructure failure as a bad proof.         |
| Receipt write outage           | 502               | Do not return a success that was not durably recorded.         |

PART VI

# Security model and production hardening

Assets, actors, trust assumptions, attack paths, current controls, residual risk, and the upgrades required for issuer-backed identity assurance.

---

01 Threat model

02 Attack analysis

03 Secrets and supply chain

04 Privacy and compliance posture

05 Production architecture roadmap

## CHAPTER 23 / QYVOX IMPLEMENTATION GUIDE

## Threat model

What Qyvox protects, who may attack it, and which systems remain trusted.

### Protected assets

| Asset                            | Why it matters                                          |
|----------------------------------|---------------------------------------------------------|
| Private birth-year witness       | Exposure defeats the data-minimization objective.       |
| Proof acceptance integrity       | False acceptance undermines relying-party policy.       |
| Circuit and artifact provenance  | A modified relation can prove the wrong statement.      |
| Service-role key                 | Compromise grants privileged database access.           |
| Anonymous access tokens          | Theft can impersonate sessions and create receipts.     |
| Verification receipts            | They encode account activity and eligibility events.    |
| Availability and bounded compute | Proof verification can be used for resource exhaustion. |

### Representative adversaries

- A dishonest user supplying a false but qualifying witness.
- A network attacker replaying, modifying, or front-running proof traffic.
- A malicious website script, dependency, extension, or compromised browser environment.
- An unauthenticated API client sending malformed, off-curve, oversized, or high-volume payloads.
- A database attacker seeking receipts or attempting to bypass replay controls.
- A compromised operator or deployment pipeline replacing circuit artifacts or configuration.
- Colluding ceremony participants retaining toxic-waste material.

### Trusted computing base

The baseline trusts the user's browser and loaded application bundle to handle the witness honestly, Circom/circomlib/SnarkJS and their build chain, the ceremony outcome, FastAPI and Node runtimes, the verification key, Supabase Auth and database, environment-secret handling, TLS termination, and the deployment operator. Zero knowledge narrows data disclosure to the verifier; it does not eliminate the rest of the system's trust assumptions.

## CHAPTER 24 / QYVOX IMPLEMENTATION GUIDE

## Attack analysis and current controls

A structured review of likely failure modes and residual risk.

### Threat-control matrix

| Vector                    | Current control                                                    | Residual risk / next step                                           |
|---------------------------|--------------------------------------------------------------------|---------------------------------------------------------------------|
| False self-asserted age   | Circuit enforces arithmetic only.                                  | Add issuer-signed credential commitment and revocation.             |
| Finite-field wraparound   | 16-bit Num2Bits on years and difference.                           | Independent circuit audit and mutation testing.                     |
| Underconstrained signal   | Explicit comparator wiring and forced output.                      | Formal constraint review and unconstrained-signal tooling.          |
| Tampered public year      | Proof binding + server UTC year check.                             | Move to exact date and signed policy version.                       |
| Exact replay              | Canonical SHA-256 fingerprint + unique DB index.                   | Add one-time challenge consumed atomically.                         |
| Front-running             | Authenticated submitter and exact replay detection.                | Bind account/session/verifier/challenge in public signals.          |
| Malformed/off-curve proof | Strict Pydantic shape; SnarkJS false verdict handling.             | Fuzz parsers and use native hardened verifier.                      |
| Resource exhaustion       | 32 KB request default, 20 s timeout, semaphore, Node memory bound. | Rate limiting, queueing, WAF, persistent verifier pool, load tests. |
| XSS/witness theft         | No intentional network serialization.                              | Strict CSP, dependency integrity, Trusted Types, security review.   |
| Service-role leakage      | Backend-only environment secret.                                   | Secret manager, rotation, scoped networks, audit alerts.            |
| Artifact substitution     | Pinned files in release tree.                                      | Signed manifest, immutable URLs, checksums, reproducible builds.    |
| Ceremony compromise       | Development contributions only.                                    | Independent public MPC with published transcript.                   |

**SECURITY POSTURE** The most consequential current risk is not breaking Groth16; it is that the witness is self-asserted. Production value depends on adding trusted provenance.

**CHAPTER 25 / QYVOX IMPLEMENTATION GUIDE**

## Secrets, dependencies, and supply-chain security

Operational controls for the parts of the system that cryptography does not protect automatically.

---

- Keep service-role and other privileged secrets in a managed secret store; never in NEXT\_PUBLIC variables, source control, build logs, client bundles, or documentation.
- Rotate any key that appears in chat, screenshots, tickets, logs, shell history, or a public artifact.
- Pin dependencies and lockfiles; review advisories for Next.js, FastAPI/Starlette, Supabase clients, SnarkJS, circomlib, and transitive packages.
- Generate software bills of materials for frontend, backend, container, and circuit toolchains.
- Sign release artifacts and container images; verify provenance in deployment.
- Use a restrictive Content Security Policy and avoid unreviewed third-party scripts on the proof page.
- Separate ceremony machines and production build machines; preserve reproducible hashes.
- Log only privacy-safe event identifiers and outcomes. Never log request bodies, authorization headers, proof coordinates at high volume, or private input.

### Container controls

The backend Dockerfile uses a multi-stage build, separates Node dependency installation from the Python runtime image, and runs the final service as a non-root qyvox user with UID 10001. Production should additionally use a read-only filesystem, drop Linux capabilities, constrain CPU/memory/PIDs, pin base-image digests, scan images, and define health checks and graceful termination.

**CHAPTER 26 / QYVOX IMPLEMENTATION GUIDE**

## Privacy, governance, and compliance posture

How to talk about regulatory value accurately and what work remains outside the code.

---

Qyvox can reduce the amount of personal information a relying party receives and stores. That may reduce breach impact, retention burden, data-subject response scope, and opportunities for secondary use. It does not automatically determine whether a deployment is lawful, exempt from identity rules, or compliant with GDPR, KYC, AML, COPPA, age-assurance, financial, gaming, or jurisdiction-specific requirements.

## Compliance boundary

| Governance question | Qyvox technical contribution                   | Still required                                                               |
|---------------------|------------------------------------------------|------------------------------------------------------------------------------|
| Data minimization   | Raw birth year omitted from API and schema.    | Document the purpose and confirm public signals are proportionate.           |
| Lawful basis        | Not determined by cryptography.                | Legal analysis, notices, consent or other basis as applicable.               |
| Retention           | Minimal three-column receipt.                  | Define retention duration and deletion workflows.                            |
| Access rights       | No DOB record to return from Qyvox.            | Account/receipt access, correction, deletion, and audit procedures.          |
| Vendor management   | Portable self-hostable architecture direction. | Contracts, subprocessors, regions, SLAs, and incident terms.                 |
| Assurance           | Public circuit and verification artifacts.     | External audit, ceremony review, penetration test, and operational evidence. |

**NOT LEGAL ADVICE** Use this architecture as evidence for counsel and compliance teams, not as a substitute for their deployment-specific analysis.

## CHAPTER 27 / QYVOX IMPLEMENTATION GUIDE

## Production architecture roadmap

The sequence that transforms the research baseline into issuer-backed, challenge-bound identity assurance.

1. **Issuer provenance.** Define a credential format whose age attribute is signed or committed by an authorized issuer. Prove signature possession or membership without revealing DOB.
2. **Exact-date policy.** Model year, month, day, leap years, jurisdiction, policy version, and trusted current time.
3. **Challenge binding.** Add nonce, session/account commitment, verifier identifier, policy identifier, and expiry as public inputs.
4. **Revocation.** Prove non-revocation or check privacy-preserving status data with bounded freshness.
5. **Ceremony.** Run and publish an independent multi-party setup tied to the audited R1CS.
6. **Verifier service.** Replace per-request Node startup with a persistent bounded pool or audited native verifier while preserving isolation and limits.
7. **Wallet and recovery.** Protect credential keys, device migration, consent, loss recovery, and multi-device use.

8. **Audit and governance.** Commission cryptographic, application, infrastructure, privacy, and legal reviews; publish resolved findings.
9. **Operations.** Add rate limits, idempotency, observability, incident response, key rotation, backup/restore tests, and SLOs.

**Suggested maturity gates**

| Phase             | Exit evidence                                                                                                    |
|-------------------|------------------------------------------------------------------------------------------------------------------|
| Research baseline | Current circuit, real browser proof, strict verifier, minimal receipt, repeatable tests.                         |
| Private alpha     | Issuer prototype, challenge binding, worker prover, persistent verifier, internal threat review.                 |
| External pilot    | Independent audit, mobile benchmark matrix, ceremony transcript, privacy assessment, monitored operations.       |
| Production        | Issuer contracts, revocation, hardened key management, SLOs, incident drills, signed releases, recurring audits. |

PART VII

# Build, run, test, and operate

A practical operator guide for local setup, environment variables, testing, deployment, observability, incident response, and troubleshooting.

---

- 01 Repository map
- 02 Local setup
- 03 Configuration reference
- 04 Testing and validation
- 05 Deployment and operations
- 06 Troubleshooting

## CHAPTER 28 / QYVOX IMPLEMENTATION GUIDE

## Repository map

Where every major component lives and how artifacts flow between directories.

---

### Current monorepo topology

```
Qyvox/
├── circuits/
│   ├── age_check.circom
│   ├── setup.sh
│   ├── smoke_test.mjs
│   └── benchmark.mjs
├── frontend/
│   ├── app/{page,whitepaper,docs,research,faq}.tsx
│   ├── components/
│   ├── lib/{zkProver,verifyProof,supabaseClient}.ts
│   └── public/zk/{age_check.wasm,age_check_final.zkey}
├── backend/
│   ├── main.py
│   ├── verify.mjs
│   ├── test_main.py
│   ├── live_integration_test.mjs
│   ├── Dockerfile
│   └── zk/verification_key.json
├── supabase/schema.sql
└── README.md
```

The circuit build produces public prover assets for the frontend and a verification key for the backend. The frontend does not need the verification key for normal proving, and the backend does not need the private witness or proving key. This separation maps cryptographic roles to deployment roles.

## CHAPTER 29 / QYVOX IMPLEMENTATION GUIDE

## Local setup and first proof

A clean-room sequence from dependencies to a working browser-to-database verification.

---

1. **Install circuit dependencies.** Run `npm ci` in `circuits` and ensure Circom 2.x and OpenSSL are available.
2. **Build artifacts.** Run `npm run setup` in `circuits`. Confirm WASM/zkey are copied to `frontend/public/zk` and `verification_key.json` to `backend/zk`.
3. **Prepare Supabase.** Enable Anonymous Sign-Ins, execute `supabase/schema.sql`, and verify browser roles cannot access `verified_users`.
4. **Configure frontend.** Set `NEXT_PUBLIC_SUPABASE_URL`, `NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY`, `NEXT_PUBLIC_API_URL`, and optionally `NEXT_PUBLIC_SITE_URL`.

5. **Configure backend.** Set SUPABASE\_URL, SUPABASE\_SERVICE\_ROLE\_KEY, and exact FRONTEND\_ORIGINS; tune verifier limits only with testing.
6. **Start API.** Install backend requirements and run Uvicorn for backend.main:app.
7. **Start portal.** Run the Next.js development server in frontend.
8. **Exercise demo.** Enter a qualifying year, inspect the real execution receipt, and confirm the request body lacks the witness.
9. **Check storage.** Verify that only user\_id, verified\_at, and proof\_hash were written.

**SECURITY SETUP** Use placeholders in documentation and .env.example files. Never copy a service-role secret into frontend/.env.local.

## CHAPTER 30 / QYVOX IMPLEMENTATION GUIDE

### Configuration reference

Every important environment control and the boundary it governs.

#### Environment-variable reference

| Variable                             | Side           | Purpose / safe guidance                                 |
|--------------------------------------|----------------|---------------------------------------------------------|
| NEXT_PUBLIC_SUPABASE_URL             | Frontend       | Project URL; safe to expose as connection metadata.     |
| NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY | Frontend       | Browser key; security depends on Auth and RLS.          |
| NEXT_PUBLIC_API_URL                  | Frontend       | Verifier base URL; use HTTPS in non-local environments. |
| NEXT_PUBLIC_SITE_URL                 | Frontend       | Canonical metadata URL.                                 |
| SUPABASE_URL                         | Backend        | Same project endpoint used by server client.            |
| SUPABASE_SERVICE_ROLE_KEY            | Backend secret | Privileged key; store and rotate as a secret.           |
| FRONTEND_ORIGINS                     | Backend        | Comma-separated exact origins; wildcard rejected.       |
| NODE_BINARY                          | Backend        | Node runtime path for isolated verifier.                |
| VERIFIER_SCRIPT                      | Backend        | Path to verify.mjs.                                     |
| VERIFICATION_KEY_PATH                | Backend        | Pinned verification_key.json path.                      |
| VERIFICATION_TIMEOUT_SECONDS         | Backend        | 1-60 second bound; default 20.                          |
| VERIFIER_CONCURRENCY                 | Backend        | Bounded parallel verifiers; default 2, maximum 16.      |
| MAX_REQUEST_BYTES                    | Backend        | Body limit; default 32,768 bytes.                       |
| EXPECTED_CURRENT_YEAR                | Backend/test   | Optional deterministic override; normally UTC year.     |

## CHAPTER 31 / QYVOX IMPLEMENTATION GUIDE

## Testing and validation

How to reproduce circuit, API, frontend, integration, and benchmark evidence.

### Validation matrix

| Layer                     | Command or suite                   | Coverage                                                                                                                  |
|---------------------------|------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| Circuit                   | npm test in circuits               | Eligible, exact boundary, underage, future year, out-of-range year, tampered public signal, backend helper.               |
| Benchmark                 | node benchmark.mjs in circuits     | 5 proving samples, 50 warm verifies, 10 subprocess verifies, payload and artifact sizes.                                  |
| Backend unit              | python -m unittest -v test_main.py | 11 focused cases using mocks; no live Supabase required.                                                                  |
| Live integration          | node live_integration_test.mjs     | Anonymous auth, real proof, RLS denial, storage columns, replay, stale/private/missing-auth/oversize/CORS cases, cleanup. |
| Frontend lint             | npm run lint                       | ESLint and Next.js rules.                                                                                                 |
| Frontend production build | npm run build                      | Compilation, TypeScript, page-data collection, static generation.                                                         |
| Manual browser QA         | Desktop and narrow viewport        | Proof flow, payload inspector, execution trace, navigation, overflow, reduced motion.                                     |

### Validation snapshot - 27 August 2026

- Circuit smoke test: passed.
- Frontend lint: passed.
- Frontend production build: passed; all public routes generated successfully.
- Fresh Node reference benchmark: prover median 104.20 ms and p95 601.69 ms across 5 samples; warm verifier median 25.68 ms and p95 45.16 ms across 50 samples; subprocess verifier median 817.90 ms and p95 1,077.76 ms across 10 samples.
- Payload and artifact sizes in that run: 724-byte proof JSON, 763-byte complete request JSON, 38,561-byte WASM, 41,244-byte proving key.
- Backend unit suite contains 11 focused tests. It was not rerun in the document-authoring runtime because the backend Python dependencies were not installed in that isolated runtime; run the documented command in the project backend environment before release.

**BENCHMARK INTERPRETATION** These timings differ from the older website research snapshot, demonstrating why benchmarks need date, environment, sample count, and percentile. Update public benchmark pages from a controlled campaign before making performance claims.

## CHAPTER 32 / QYVOX IMPLEMENTATION GUIDE

## Deployment, observability, and incident response

The operational practices needed to run the verifier safely.

### Deployment checklist

- Terminate TLS at a trusted edge and redirect HTTP to HTTPS.
- Set exact production CORS origins and verify denied preflights.
- Keep the service-role key in a secret manager and restrict who can read or rotate it.
- Run the backend container as non-root with a read-only filesystem and resource limits.
- Pin the circuit release manifest and verification key in the deployed image.
- Apply edge and account rate limits before expensive verification.
- Use a bounded persistent verifier pool or audited native verifier for predictable latency.
- Monitor health, dependency status, rejection classes, queue depth, verifier timeouts, database latency, and receipt conflicts without logging proof bodies or tokens.
- Test backup/restore and receipt-retention deletion procedures.

### Incident priorities

#### Incident-response playbook

| Incident               | Immediate action                                                | Follow-up                                                                |
|------------------------|-----------------------------------------------------------------|--------------------------------------------------------------------------|
| Service-role exposure  | Revoke/rotate; disable affected service; audit database access. | Identify leak path, narrow permissions, notify stakeholders as required. |
| Artifact mismatch      | Stop acceptance; roll back to known manifest.                   | Rebuild, verify hashes, investigate pipeline, reissue release evidence.  |
| Verifier vulnerability | Disable or isolate affected path; deploy patched verifier.      | Revalidate proofs/receipts per risk assessment and publish advisory.     |
| Database outage        | Maintain fail-closed behavior; communicate degraded status.     | Restore service, reconcile attempts, test replay uniqueness.             |
| Frontend compromise    | Take proof page offline; revoke sessions if necessary.          | Audit scripts, CSP, dependencies, and potential witness exposure.        |

## CHAPTER 33 / QYVOX IMPLEMENTATION GUIDE

## Troubleshooting

A symptom-to-cause guide for the most likely setup and runtime failures.

### Troubleshooting matrix

| Symptom                                   | Likely cause                                                                                     | Resolution                                                                                                               |
|-------------------------------------------|--------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| "Qyvox authentication is not configured." | Missing frontend Supabase URL or publishable key at build/runtime.                               | Set NEXT_PUBLIC variables, restart Next.js, inspect browser bundle configuration; never substitute the service-role key. |
| Private sign-in disabled                  | Anonymous sign-ins are off in Supabase Auth.                                                     | Enable Anonymous Sign-Ins and retry with a fresh session.                                                                |
| Replay protection temporarily unavailable | verified_users query failed: schema, credentials, network, RLS/service-role, or Supabase outage. | Check /status, backend logs without secrets, schema deployment, service-role configuration, network and project status.  |
| Verifier temporarily unavailable          | Node missing, verify.mjs/vkey missing, timeout, nonzero subprocess, malformed stdout.            | Confirm image contents, paths, Node runtime, artifact manifest, memory and timeout.                                      |
| Proof invalid                             | Wrong artifacts, public signals, witness, or tampered/off-curve points.                          | Rebuild one artifact set, verify public order/year, reproduce circuit smoke test.                                        |
| 403 current period                        | publicSignals are not [1, server UTC year].                                                      | Use UTC year and a qualifying witness; check time/config override.                                                       |
| 422 body validation                       | Extra/private field, scalar formatting, wrong coordinate dimensions, protocol/curve mismatch.    | Send the exact proof-only schema.                                                                                        |
| CORS preflight rejected                   | Origin absent or has mismatched scheme/host/port/trailing configuration.                         | Add exact origin to FRONTEND_ORIGINS and restart API.                                                                    |
| Duplicate/409                             | Account already has receipt or proof hash was consumed.                                          | Treat as expected idempotency/replay outcome; decide product recovery policy.                                            |
| Mobile tab stalls                         | Main-thread proving or memory/thermal pressure.                                                  | Move proving to worker, show progress, reduce constraints/artifacts, benchmark target devices.                           |

PART VIII

# Product, business, and research strategy

How the architecture creates value, which customers benefit, how to evaluate adoption, and what evidence should guide the next research phase.

---

01 Value proposition

02 Enterprise adoption

03 Research and benchmarking

04 Open source and assurance

## CHAPTER 34 / QYVOX IMPLEMENTATION GUIDE

## Product and business value

How to translate the architecture into a credible outcome for buyers, investors, and grant reviewers.

Qyvox's product value is not "cryptography for its own sake." It is the ability to separate an eligibility decision from custody of the underlying personal fact. This can reduce the data stored by a relying party, improve user trust, narrow breach consequences, and make privacy a verifiable system property rather than a privacy-policy promise.

### Stakeholder value map

| Audience                 | Primary value                                                          | Evidence they need                                                                          |
|--------------------------|------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| Enterprise buyer         | Reduced PII custody with a familiar API integration.                   | Threat model, deployment model, issuer plan, audit results, SLAs, legal assessment.         |
| Developer/CTO            | Portable browser prover and strict proof-only endpoint.                | Quickstart, SDK ergonomics, versioned artifacts, tests, performance, self-hosting.          |
| Investor/grant committee | Deep-tech differentiation and defensible privacy architecture.         | Technical proof of work, market problem, milestones, independent validation, adoption path. |
| End user                 | Eligibility without uploading a birth year or identity image to Qyvox. | Clear UX, trustworthy issuer, transparent disclosure, recovery and support.                 |
| Auditor/researcher       | Public relation and measurable trust boundaries.                       | Source, R1CS, ceremony transcript, manifests, test corpus, findings and remediation.        |

## Responsible positioning

The honest near-term positioning is "privacy-preserving eligibility infrastructure and research baseline," not "complete identity truth." The strongest commercialization milestone is an issuer-backed credential flow that preserves the current proof-only relying-party boundary.

## CHAPTER 35 / QYVOX IMPLEMENTATION GUIDE

## Enterprise adoption and integration

A practical evaluation sequence for prospective relying parties and self-hosting teams.

1. **Define the predicate.** Identify the minimum fact the business truly needs and the policy/jurisdiction governing it.

2. **Choose issuer trust.** Decide which authorities can attest the underlying attribute and how revocation works.
3. **Model public signals.** Expose only the necessary result, policy, verifier, challenge, and expiry fields.
4. **Integrate SDK.** Run proving locally and ensure application analytics never capture witness fields.
5. **Integrate API.** Authenticate, issue challenges, verify proofs, and consume challenges atomically.
6. **Design receipts.** Store only identifiers, timestamps, policy references, and fingerprints required for accountability.
7. **Audit.** Review circuit, ceremony, frontend supply chain, backend, infrastructure, privacy, and legal controls.
8. **Pilot.** Measure completion, error rate, mobile performance, support burden, fraud outcomes, and user trust.

#### Enterprise due-diligence summary

| Adoption question                | Current answer                                                                                     |
|----------------------------------|----------------------------------------------------------------------------------------------------|
| Blockchain required?             | No. Browser WASM, REST API, and PostgreSQL are sufficient.                                         |
| Can verifier be self-hosted?     | Architecture supports it; production packaging, manifests, support, and terms remain roadmap work. |
| Can Qyvox recover DOB?           | The baseline never receives DOB/birthYear; proof privacy is computational under assumptions.       |
| Does it prove truth?             | Not until a trusted issuer credential is included.                                                 |
| Does it reduce compliance scope? | Potentially, through minimization; exact effect is jurisdiction and deployment specific.           |
| Can one proof be reused?         | Exact reuse is blocked now; challenge binding is required for stronger freshness.                  |

## CHAPTER 36 / QYVOX IMPLEMENTATION GUIDE

### Research and benchmarking program

How to produce evidence that is comparable, honest, and useful for engineering decisions.

The current circuit is intentionally small, so performance conclusions must not be generalized to issuer signatures, revocation proofs, or richer date logic. Every public benchmark should publish hardware, operating system, browser/runtime, power state, artifact cache state, sample counts, warmup, median, p95/p99, failure rate, memory, and artifact sizes.

## Research campaign matrix

| Campaign                   | Measurements                                                                          | Decision enabled                                         |
|----------------------------|---------------------------------------------------------------------------------------|----------------------------------------------------------|
| Mobile prover matrix       | Cold/warm p50/p95, peak memory, thermal drift, crash rate, artifact download.         | Worker strategy, circuit budget, supported device floor. |
| Verifier load test         | Throughput, p50/p95/p99, queue depth, CPU/RAM, timeout/error rate.                    | Pool size, autoscaling, rate limits, SLOs.               |
| Network impairment         | 3G/4G latency, loss, cache misses, retry behavior.                                    | Payload delivery and UX resilience.                      |
| Circuit growth             | Constraints, zkey/WASM size, proving and verification deltas per feature.             | Issuer/revocation design tradeoffs.                      |
| Security failure injection | DB outage, stale challenge, verifier crash, malformed points, concurrency exhaustion. | Fail-closed confidence and incident readiness.           |
| User study                 | Completion, comprehension, trust, abandonment, support questions.                     | Disclosure language and interface improvements.          |

**CURRENT BENCHMARK** The 27 August 2026 reference run is a Node measurement on one environment. It is evidence that the pipeline works—not proof of mobile performance or a production latency guarantee.

## CHAPTER 37 / QYVOX IMPLEMENTATION GUIDE

## Open source, auditability, and trust

How public artifacts can convert technical claims into independently reproducible evidence.

- Publish circuit source, R1CS, symbol map, WASM, proving key, verification key, and cryptographic hashes as one release.
- Publish ceremony transcripts, contributor verification, and the exact circuit hash used during setup.
- Publish negative tests, benchmark scripts, test vectors, and expected public-signal ordering.
- Maintain a security policy, vulnerability-reporting channel, disclosure timeline, and signed advisories.
- Separate open-source SDK/circuit licensing from commercial orchestration only with clear, reviewable terms.
- Track external audit findings and remediation status without presenting a point-in-time audit as permanent safety.
- Expose a live status page that distinguishes API, verifier, and database readiness without leaking internals.

A polished portal supports reproducible trust, but it cannot replace published source, artifact hashes, test vectors, and deployment-key verification.

PART IX

# Reference appendices

Compact source-of-truth material for integration, review, operations, and first-time learning.

---

01 API examples

02 Glossary

03 Frequently asked questions

04 Release checklist

05 Source-of-truth index

## CHAPTER A / QYVOX IMPLEMENTATION GUIDE

## API examples

Copyable request and response patterns with private fields deliberately absent.

## Illustrative authenticated verification request

```
curl -X POST "$QYVOX_API_URL/api/v1/verify" \
  -H "Authorization: Bearer $SUPABASE_ACCESS_TOKEN" \
  -H "Content-Type: application/json" \
  --data-binary @proof-bundle.json
```

## Successful response shape; durations are illustrative

```
{
  "verified": true,
  "verified_at": "2026-08-27T00:00:00Z",
  "trace": {
    "proof_system": "Groth16",
    "curve": "BN254 / bn128",
    "received_fields": ["proof", "publicSignals"],
    "stored_fields": ["user_id", "verified_at", "proof_hash"],
    "steps": [
      {"component": "policy", "outcome": "passed", "ms": 0.1},
      {"component": "replay", "outcome": "passed", "ms": 12.3},
      {"component": "verifier", "outcome": "passed", "ms": 800.0},
      {"component": "database", "outcome": "committed", "ms": 20.0}
    ],
    "total_ms": 832.4
  }
}
```

**PROHIBITED BODY FIELD** A request that adds birthYear, dob, age, user\_id, or any unknown top-level field fails strict validation. The authorization token belongs in the header, not the JSON body.

## CHAPTER B / QYVOX IMPLEMENTATION GUIDE

## Glossary

Plain-language definitions for the cryptography, web, database, and security terms used throughout Qyvox.

## Qyvox glossary

| Term                     | Definition                                                                                     |
|--------------------------|------------------------------------------------------------------------------------------------|
| Anonymous authentication | A session with a stable account identifier but no required name, email, or profile attributes. |
| Artifact manifest        | A signed or published record binding source, circuit, keys, and hashes into one release.       |
| BN254 / bn128            | The pairing-friendly elliptic curve used by the current SnarkJS Groth16 setup.                 |

| Term                    | Definition                                                                                                   |
|-------------------------|--------------------------------------------------------------------------------------------------------------|
| Canonical serialization | A deterministic JSON representation used so equivalent payloads produce the same fingerprint.                |
| Challenge               | A fresh server-issued value that a proof can bind to for stronger replay and front-running resistance.       |
| Circom                  | A language for describing arithmetic constraints used by zero-knowledge proving systems.                     |
| Circuit                 | The mathematical relation that decides whether witness and public signals are valid.                         |
| Completeness            | Valid witnesses can produce proofs accepted by an honest verifier.                                           |
| Constraint              | A field equation that every valid witness assignment must satisfy.                                           |
| CORS                    | A browser policy controlling which origins may call an API; not an authentication mechanism.                 |
| Credential commitment   | A cryptographic representation that can bind hidden credential attributes to an issuer statement.            |
| Data minimization       | Collecting, using, and retaining only information necessary for a purpose.                                   |
| Elliptic-curve point    | A group element used in proof construction and pairing-based verification.                                   |
| Fail closed             | Reject or stop when a security dependency is unavailable rather than bypassing the control.                  |
| Finite field            | A bounded arithmetic system in which operations wrap modulo a large prime.                                   |
| Front-running           | Submitting another party's observed transaction or proof first to capture its effect.                        |
| Groth16                 | A succinct non-interactive argument system with a circuit-specific trusted setup.                            |
| JWT / bearer token      | A credential presented in an HTTP Authorization header; possession grants the represented session authority. |
| Knowledge soundness     | A successful prover is understood to know a witness satisfying the relation, under formal assumptions.       |
| Nonce                   | A value intended for one use, commonly added to make a proof fresh.                                          |
| Num2Bits                | A Circom gadget that constrains a value to a specific non-negative bit width.                                |
| Pairing                 | A bilinear map used by Groth16 verification to check algebraic relationships efficiently.                    |
| PII                     | Personally identifiable information; exact legal definitions vary.                                           |
| Predicate               | A yes/no statement such as age eligibility.                                                                  |
| Proof                   | Public cryptographic evidence that a witness satisfying the circuit exists.                                  |
| Proof fingerprint       | Qyvox's SHA-256 digest of the canonical proof bundle, used to reject exact reuse.                            |
| Proving key             | Public circuit-specific material used with the witness to generate a Groth16 proof.                          |
| Public signal           | A value intentionally disclosed and cryptographically bound to the proof.                                    |
| Powers of Tau           | A reusable multi-party setup phase used by pairing-based SNARK workflows.                                    |
| R1CS                    | Rank-1 Constraint System, the multiplication-of-linear-combinations representation of circuit constraints.   |
| Replay attack           | Reusing a valid message or proof outside its intended one-time context.                                      |

| Term                        | Definition                                                                                                           |
|-----------------------------|----------------------------------------------------------------------------------------------------------------------|
| Revocation                  | Invalidating a credential before its nominal expiry.                                                                 |
| RLS                         | PostgreSQL row-level security, which controls database-row access per role and policy.                               |
| SNARK                       | Succinct Non-interactive Argument of Knowledge.                                                                      |
| Soundness                   | Invalid statements should not be accepted except with negligible probability under assumptions.                      |
| Structured reference string | Public parameters produced by setup and used by the prover and verifier.                                             |
| Toxic waste                 | Secret setup randomness that must not remain available to a colluding adversary.                                     |
| Verification key            | Public circuit-specific material used to verify Groth16 proofs.                                                      |
| Verifier                    | The algorithm or service that checks a proof against public signals and a verification key.                          |
| WASM / WebAssembly          | Portable bytecode executed by the browser for the witness calculation.                                               |
| Witness                     | Private and intermediate values assigned to circuit signals for one proof.                                           |
| Zero knowledge              | A property that the proof reveals no witness information beyond the public statement under the scheme's assumptions. |

## CHAPTER C / QYVOX IMPLEMENTATION GUIDE

### Frequently asked questions

Short, review-ready answers that preserve the current implementation boundary.

---

#### Can Qyvox recover a user's birth year?

The server never receives it. Recovering the witness from a valid proof should be computationally infeasible under Groth16 assumptions; this is not a claim of information-theoretic impossibility.

#### Does Qyvox need a blockchain?

No. The baseline uses browser WebAssembly, a REST API, and PostgreSQL.

#### Does the proof prove the year is true?

No. It proves knowledge of a qualifying value. Trusted issuer provenance is the principal production upgrade.

#### What is stored?

Authenticated user UUID, verification timestamp, and exact-proof SHA-256 fingerprint.

### Why is currentYear public?

It binds the proof to a policy epoch the server can check.

### Why is isEligible public?

The relying party needs the yes/no result; the circuit forces it to one.

### Can a proof be replayed?

Exact payload reuse is blocked. Fresh randomized proofs and front-running require challenge/session binding for stronger prevention.

### What if Supabase is down?

Replay and persistence failures return 502; the service does not claim success.

### What if the verifier crashes?

The API returns 503 rather than classifying the proof as invalid.

### Are the proving key and WASM secret?

No. They are public artifacts, but their integrity and version provenance remain critical.

### Is the setup production-ready?

No. The included ceremony is a reproducible development ceremony on one machine.

### Does clearing React state erase RAM?

It removes application references promptly but cannot guarantee physical zeroization in a managed browser runtime.

### Is Qyvox compliant by default?

No. Data minimization helps, but legal and operational obligations depend on jurisdiction and deployment.

## CHAPTER D / QYVOX IMPLEMENTATION GUIDE

### Release and audit checklist

A single gate for cryptography, application, infrastructure, privacy, and evidence before external use.

- 
- ☐ Circuit policy specification approved by product, legal, and cryptography reviewers.

- ☐ No unconstrained or underconstrained security-relevant signals found.
- ☐ Negative, boundary, property, mutation, and cross-implementation tests pass.
- ☐ Independent circuit and verifier audit findings resolved or explicitly accepted.
- ☐ Issuer signature/commitment and revocation model implemented for production claims.
- ☐ One-time challenge, session/verifier binding, policy identifier, and expiry enforced.
- ☐ Public MPC ceremony complete; transcripts and artifact hashes published.
- ☐ Frontend artifacts use immutable version binding and strong supply-chain controls.
- ☐ Birth year absent from requests, logs, analytics, traces, error reports, and database schemas.
- ☐ Service-role and signing secrets held in managed secret systems and rotation tested.
- ☐ RLS, grants, and direct browser access validated in a live project.
- ☐ Rate limits, body limits, timeouts, concurrency caps, and resource limits load-tested.
- ☐ Mobile prover matrix completed with memory, thermal, latency, and failure metrics.
- ☐ CORS, CSP, TLS, dependency, container, and penetration-testing controls verified.
- ☐ Monitoring and alerting avoid private payload capture and are tested under failure injection.
- ☐ Retention, deletion, incident response, backup/restore, and user-rights procedures approved.
- ☐ Legal/compliance review completed for each target jurisdiction and use case.
- ☐ Release manifest signed; deployed WASM, zkey, and verification key match it.
- ☐ User-facing copy states capabilities, assumptions, and limitations accurately.

## CHAPTER E / QYVOX IMPLEMENTATION GUIDE

### Source-of-truth index

The implementation files a reviewer should inspect first.

#### Implementation evidence index

| Question                             | Primary source                                   |
|--------------------------------------|--------------------------------------------------|
| What relation is proved?             | circuits/age_check.circom                        |
| How are artifacts generated?         | circuits/setup.sh                                |
| Which adversarial cases are tested?  | circuits/smoke_test.mjs and backend/test_main.py |
| How are performance values measured? | circuits/benchmark.mjs                           |
| How is the proof created locally?    | frontend/lib/zkProver.ts                         |

| Question                                    | Primary source                            |
|---------------------------------------------|-------------------------------------------|
| When is the witness cleared?                | frontend/components/ProofDemo.tsx         |
| What body is sent?                          | frontend/lib/verifyProof.ts               |
| How is anonymous auth configured?           | frontend/lib/supabaseClient.ts            |
| What does the API accept and enforce?       | backend/main.py                           |
| How is SnarkJS isolated?                    | backend/verify.mjs and backend/Dockerfile |
| What is stored?                             | supabase/schema.sql                       |
| What does the live end-to-end test cover?   | backend/live_integration_test.mjs         |
| What are known limitations and setup steps? | README.md, /whitepaper, /docs, /faq       |

## Final perspective

Qyvox already demonstrates the most important architectural inversion: the sensitive witness stays on the user side while the verifier receives a proof of the policy fact. The engineering baseline is credible because the circuit, proof generation, strict API, replay store, receipt schema, and execution trace can be inspected and tested. Its next stage is equally clear: add issuer-backed provenance, exact-date policy, cryptographic freshness, an independent ceremony, hardened operations, and external audit without surrendering the proof-only server boundary.

**QYVOX** Prove the fact. Keep the data.